WEB APPLICATION FIREWALL USING CLOUDFLARE

Minor project report submitted
in partial fulfillment of the requirement for award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

By

M.SAMPATH	(20UECS0553)	**(17625)**
P.ARUL SIVA	(20UECS0674)	**(17237)**
CH.LAKSHMI PRASAD	(20UECS0171)	**(17362)**

Under the guidance of
R. ANTO PRAVIN, M.E.,
ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE & TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)
Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA

May, 2023

WEB APPLICATION FIREWALL USING CLOUDFLARE

Minor project report submitted
in partial fulfillment of the requirement for award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

By

M.SAMPATH	(20UECS0553)	**(17625)**
P.ARUL SIVA	(20UECS0674)	**(17237)**
CH.LAKSHMI PRASAD	(20UECS0171)	**(17362)**

Under the guidance of
R. ANTO PRAVIN, M.E.,
ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE & TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)
Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA

May, 2023

CERTIFICATE

It is certified that the work contained in the project report titled ”WEB APPLICATION FIREWALL USING CLOUDFLARE)” by ”M.SAMPATH (20UECS0553), P.ARUL SIVA (20UECS0674), LAKSHMI PRASAD.CH (20UECS0171)” has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor
R. Anto Pravin
Assistant Professor
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science & Technology
May, 2023

Signature of Head of the Department
Dr. M. S.Muralidhar., M.E., Ph.D.
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science & Technology
May, 2023

Signature of the Dean
Dr. V. Srinivasa Rao
Professor & Dean
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science & Technology
May, 2023

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
M.SAMPATH
Date: / /

(Signature)
P.ARUL SIVA
Date: / /

(Signature)
CH.LAKSHMI PRASAD
Date: / /

APPROVAL SHEET

This project report entitled ”WEB APPLICATION FIREWALL USING CLOUDFLARE” by M. SAMPATH (20UECS0553), P. ARUL SIVA (20UECS0674), CH. LAKSHMI PRASAD (20UECS0171) is approved for the degree of B.Tech in Computer Science & Engineering.

Examiners **Supervisor**

R. Anto Pravin , M.E.,

Date: / /

Place:

ACKNOWLEDGEMENT

We express our deepest gratitude to our respected **Founder Chancellor and President Col. Prof. Dr. R. RANGARAJAN B.E. (EEE), B.E. (MECH), M.S (AUTO),D.Sc., Foundress President Dr. R. SAGUNTHALA RANGARAJAN M.B.B.S.** Chairperson Managing Trustee and Vice President.

We are very much grateful to our beloved **Vice Chancellor Prof. S. SALIVAHANAN,** for providing us with an environment to complete our project successfully.

We record indebtedness to our **Professor & Dean, Department of Computer Science & Engineering, School of Computing, Dr. V. SRINIVASA RAO, M.Tech., Ph.D.,** for immense care and encouragement towards us throughout the course of this project.

We are thankful to our **Head, Department of Computer Science & Engineering, Dr. M.S. MURALI DHAR, M.E., Ph.D.,** for providing immense support in all our endeavors.

We also take this opportunity to express a deep sense of gratitude to our Internal Supervisor **Mr. R. ANTO PRAVIN, M.E.,** for his cordial support, valuable information and guidance, he helped us in completing this project through various stages.

A special thanks to our **Project Coordinators Mr. V. ASHOK KUMAR, M.Tech., Ms. C. SHYAMALA KUMARI, M.E.,** for their valuable guidance and support throughout the course of the project.

We thank our department faculty, supporting staff and friends for their help and guidance to complete this project.

M.SAMPATH	**(20UECS0553)**
P.ARUL SIVA	**(20UECS0674)**
CH. LAKSHMI PRASAD	**(20UECS0171)**

ABSTRACT

A Web Application Firewall (WAF) is a security tool created to guard online applications against many forms of assaults, such as cross-site scripting, SQL injection, and other flaws. Between the web server and the client, it functions as an intermediate, removing harmful traffic and enforcing security regulations to stop attacks. To protect the integrity and accessibility of online services, WAFs identify and thwart threats in real-time. This highlights the crucial function of WAFs in protecting web applications from attacks and their advantages for businesses. It works by examining online traffic and removing requests or answers that might be hazardous, so preventing unauthorised access to web servers and sensitive data. A WAF can be built as hardware or software and deployed in a variety of places, including as part of a load balancer or in front of web servers, for example. A WAF is fundamental to the security of online applications and their defence against the continuously changing threat landscape.

Keywords: Attacks, Cross-Site Scripting, SQL injection, security, Web Application Firewall(WAF), web servers, vulnerabilities.

LIST OF FIGURES

4.1 **Reverse Proxy Server** 12
4.2 **Data Flow Diagram** 13
4.3 **Use Case Diagram** 14
4.4 **Class Diagram** 15
4.5 **Sequence Diagram** 16
4.6 **Collaboration Diagram** 17
4.7 **Activity Diagram** 18

5.1 **Input Design** 24
5.2 **Output Design** 25
5.3 **Unit Testing** 26
5.4 **Integration Testing** 27
5.5 **System Testing** 28
5.6 **Result** 29

6.1 **WAF** 35
6.2 **WAF Detection Tools** 36

8.1 **Plagiarism Report** 39

LIST OF ACRONYMS AND ABBREVIATIONS

CSS Cross-Site Scripting

GDPR General Data Protrction Regulation

HIPAA Health Insurance Portablilty and Accountability Act

HTTP Hyper Text Transfer Protocol

IDS Intrusion Detection System

SSD Solid State Drive

SIEM Security Information and Event Management

SP Security Protection

PCIDSS Paytment Card Industry Data Security Standard

WAF Web Application Firewall

TABLE OF CONTENTS

Page.No

ABSTRACT **v**

LIST OF FIGURES **vi**

1 INTRODUCTION **1**

1.1 Introduction 1
1.2 Aim of the Project 2
1.3 Project Domain 2
1.4 Scope of the Project 3

2 LITERATURE REVIEW **4**

3 PROJECT DESCRIPTION **7**

3.1 Existing System 7
3.2 Proposed System 7
3.3 Feasibility Study 8
3.3.1 Economic Feasibility 8
3.3.2 Technical Feasibility 8
3.3.3 Social Feasibility 9
3.4 System Specification 9
3.4.1 Hardware Specification 9
3.4.2 Software Specification 10
3.4.3 Standards and Policies 11

4 METHODOLOGY **12**

4.1 Reverse Proxy Server 12
4.2 Design Phase 13
4.2.1 Data Flow Diagram 13
4.2.2 Use Case Diagram 14
4.2.3 Class Diagram 15
4.2.4 Sequence Diagram 16

4.2.5 Collaboration Diagram 17
4.2.6 Activity Diagram 18
4.3 Algorithm & Pseudo Code 19
4.3.1 Algorithm 19
4.3.2 Pseudo Code 19
4.4 Module Description 20
4.4.1 Traffic Interception Module 20
4.4.2 Rule Engine Module 21
4.4.3 Logging and Reporting Module 22
4.5 Steps to execute/run/implement the Project 23
4.5.1 Data Preparation 23
4.5.2 Model Training 23
4.5.3 Vulnerability Detection 23

5 IMPLEMENTATION AND TESTING 24
5.1 Input and Output 24
5.1.1 Input Design 24
5.1.2 Output Design 25
5.2 Testing 26
5.3 Types of Testing 26
5.3.1 Unit Testing 26
5.3.2 Integration Testing 27
5.3.3 System Testing 28
5.3.4 Test Result 29

6 RESULTS AND DISCUSSIONS 30
6.1 Efficiency of the Proposed System 30
6.2 Comparison of Existing and Proposed System 30
6.3 Sample Code 32

7 CONCLUSION AND FUTURE ENHANCEMENTS 37
7.1 Conclusion 37
7.2 Future Enhancements 37

8 PLAGIARISM REPORT 39

9 SOURCE CODE & POSTER PRESENTATION **40**
9.1 Source Code 40
9.2 Poster Presentation 44

References **44**

Chapter 1

INTRODUCTION

1.1 Introduction

A Web Application Firewall (WAF) is a security solution designed to protect web applications and websites from various types of cyber threats, including attacks such as cross-site scripting, SQL injection, and other application-layer attacks. A WAF works by analyzing incoming web traffic to a web application and blocking any malicious traffic that matches a predefined set of rules. This can be done through various techniques such as signature-based detection, behavioral analysis, and anomaly detection. WAFs can be deployed on-premises or in the cloud and can be configured to provide different levels of protection depending on the needs of the organization. Some WAFs can be integrated with other security solutions such as IDS and SIEM tools for better threat detection and response. Overall, a WAF is an important component of a comprehensive web application security strategy, helping organizations to protect their web assets from cyber threats and ensure the security and privacy of their users.

A WAF is a type of security tool that is designed to protect web applications from various types of cyberattacks. It is a software or hardware solution that sits between the web application and the internet, analyzing incoming traffic and blocking any potentially malicious requests. Web applications are a common target for cyberattacks as they often handle sensitive user data such as login credentials and payment information. Hackers can exploit vulnerabilities in web applications to steal this data or to gain access to a network or system. A WAF can help prevent these attacks by filtering traffic and blocking malicious requests. It does this by using a set of rules to identify and block known attack vectors such as SQL injection, cross-site scripting, and other common web-based attacks. A WAF is a type of security tool that is designed to protect web applications from various types of cyberattacks. It is a software or hardware solution that sits between the web application and the internet, analyzing incoming traffic and blocking any potentially malicious requests. Web applications are a common target for cyberattacks as they often handle sensitive user data such

as login credentials and payment information. Hackers can exploit vulnerabilities in web applications to steal this data or to gain access to a network or system. A WAF can help prevent these attacks by filtering traffic and blocking malicious requests. It does this by using a set of rules to identify and block known attack vectors such as SQL injection, cross-site scripting, and other common web-based attacks.

1.2 Aim of the Project

The main aim of a WAF is to protect web applications and websites from a variety of attacks that target the application layer. Web applications are a common target for cybercriminals seeking to exploit vulnerabilities in software and gain unauthorized access to sensitive data or resources. These attacks can lead to data breaches, system downtime and other security incidents that can have serious consequences for organizations. WAF is to provide an additional layer of security for web applications, protecting against cyber attacks and vulnerabilities that could lead to data breaches, downtime, and other costly incidents.

1.3 Project Domain

The project domain of WAF involves the implementation and management of a WAF solution for web applications and websites. This includes the selection of an appropriate WAF solution based on the organization's needs and budget, the installation and configuration of the solution, and the ongoing monitoring and maintenance of the system. The project domain also involves identifying potential vulnerabilities and threats to web applications and developing a set of security policies and rules to protect against these threats. This may involve conducting vulnerability assessments and penetration testing to identify weaknesses in the application layer.
In addition, the project domain of a WAF may include integrating the solution with other security tools and technologies such as IDS and SIEM tools to provide a comprehensive security posture. Overall, the project domain of a WAF involves ensuring the security and availability of web applications and websites, protecting sensitive data and resources, and ensuring compliance with regulatory requirements related to web application security.

1.4 Scope of the Project

The WAF is vast and covers a wide range of industries and applications. WAFs are used in various sectors, including finance, healthcare, e-commerce, government, and more, to protect web applications from cyber threats such as SQL injection, cross-site scripting, and other application-layer attacks.The scope of a WAF includes the ability to detect and block malicious traffic, preventing unauthorized access to sensitive data, and ensuring the confidentiality and integrity of information. In the WAFs refers to the specific area or scope in which a WAF will be implemented or deployed. This can vary widely depending on the organization's needs and requirements.
The WAF also includes compliance with regulatory requirements related to web application security, such as the PCI DSS and the HIPAA. Overall, the scope of a WAF is to provide a comprehensive security solution for web applications and websites, protecting them from cyber threats, ensuring compliance with regulations, and maintaining the security and privacy of users.

Chapter 2

LITERATURE REVIEW

Sohail et al. (2021) [1] conducted a comprehensive review of web application firewalls and their effectiveness in protecting web applications against various attacks. The analyzed the key features, architecture, and deployment models of WAFs, along with their advantages and limitations. The identified that WAFs can be deployed in several ways, such as on-premise, cloud-based, or hybrid models. They also found that WAFs can be categorized based on their detection mechanisms, such as signature-based, anomaly-based, and hybrid approaches.

Agarwal et al. (2021) [2] presented a comparative analysis of various WAF solutions available in the market. The evaluated several WAF products, including open-source solutions, cloud-based WAFs, and proprietary solutions. The found that open-source WAF solutions are cost-effective but may not provide the same level of features and support as proprietary solutions. Cloud-based WAFs were found to be effective in providing scalable solutions with ease of deployment, but they may not be suitable for organizations with strict data privacy and compliance requirements.

Alghamdi et al. (2020) [3] developed a machine learning-based WAF solution using a novel deep learning architecture to detect and prevent web application attacks. The evaluated the proposed solution's performance and compared it with traditional rule-based and machine learning-based WAF solutions. The used a deep learning architecture called Multi-Branch Convolutional Neural Network (MB-CNN) to train the WAF solution to detect and prevent attacks based on patterns and anomalies in web traffic. The MB-CNN architecture includes multiple parallel convolutional neural networks that process different types of features in web traffic, such as HTTP headers and payloads.

R. S. Alhakami, et al. (2021) [4] offered a machine learning-based WAF to improve the effectiveness of detecting and preventing web application attacks. The evaluated the proposed solution's performance and compared it with traditional rule-

based WAF solutions. The used machine learning algorithms to train the WAF solution to detect and prevent attacks based on patterns and anomalies in web traffic. They evaluated the proposed solution's effectiveness in detecting common web application attacks, such as SQL injection and cross-site scripting.

S. S. Gujrathi et al. (2020) [5] developed a hybrid WAF solution that combines signature-based and anomaly-based approaches to improve the effectiveness of detecting and preventing web application attacks. The evaluated the proposed hybrid WAF solution's effectiveness in detecting and preventing common web application attacks, such as SQL injection and cross-site scripting, and compared it with traditional signature-based and anomaly-based WAF solutions.

N. M. Alharthi et al. (2020) [6] conducted a deep learning-based WAF solution to improve the effectiveness of detecting and preventing web application attacks. The evaluated the proposed solution's performance and compared it with traditional rule-based and machine learning-based WAF solutions. The used deep learning algorithms, specifically convolutional neural networks (CNNs), to train the WAF solution to detect and prevent attacks based on patterns and anomalies in web traffic. They evaluated the proposed solution's effectiveness in detecting common web application attacks, such as SQL injection and cross-site scripting.

Y. A. Al-Dujaili, et al. (2019) [7] presented a hybrid WAF solution that combines machine learning and signature-based approaches to improve the effectiveness of detecting and preventing web application attacks. The evaluated the proposed solution's performance and compared it with traditional rule-based and machine learning-based WAF solutions.The used machine learning algorithms, specifically the k-nearest neighbors (k-NN) algorithm, to train the WAF solution to detect and prevent attacks based on patterns and anomalies in web traffic. They also used a signature-based approach to detect known attack patterns.

S. S. Suhail et al. (2020) [8] developed a rule-based WAF solution using Apache Struts to prevent web application attacks. The evaluated the proposed solution's performance and compared it with traditional rule-based and machine learning-based WAF solutions. The used the Apache Struts framework, which provides a set of built-in rules for detecting and preventing common web application attacks such as

SQL injection and cross-site scripting. They also added custom rules to the WAF solution to detect attacks specific to their use case.

M. A. B. A. Rahman et al. (2018) [9] conducted a hybrid WAF solution that combines rule-based and machine learning-based approaches to improve the effectiveness of detecting and preventing web application attacks. The evaluated the proposed solution's performance and compared it with traditional rule-based and machine learning-based WAF solutions. The used machine learning algorithms, specifically the decision tree algorithm, to train the WAF solution to detect and prevent attacks based on patterns and anomalies in web traffic. They also used a rule-based approach to detect known attack patterns.

P. Khanna et al. (2021) [10] presented a hybrid WAF solution using a combination of rule-based and machine learning-based approaches to improve the accuracy and performance of web application attack detection and prevention. The evaluated the proposed solution's effectiveness and compared it with traditional rule-based and machine learning-based WAF solutions. The used a rule-based approach to detect known attack patterns and a machine learning-based approach using Support Vector Machine (SVM) algorithm to detect anomalous patterns in web traffic. The proposed hybrid WAF solution combines the strengths of both approaches and addresses the limitations of each.

Chapter 3

PROJECT DESCRIPTION

3.1 Existing System

A WAF is a security tool designed to protect web applications from a range of attacks, such as SQL injection, cross-site scripting, and session hijacking. The existing system of WAF typically involves a set of rules and policies that are applied to incoming web traffic to identify and block potential threats. WAFs can be implemented as software or hardware solutions and can be deployed on-premises or in the cloud. They work by examining the contents of each HTTP request and response and comparing them to predefined rules. If a request or response matches a rule, the WAF will block it or take other protective measures.

One disadvantage of WAFs is that they can be complex to set up and maintain. The rules and policies must be carefully configured to avoid blocking legitimate traffic or allowing malicious traffic through. Additionally, WAFs can introduce latency and may impact application performance if not properly tuned. Another disadvantage is that WAFs are not foolproof. Attackers can use evasion techniques, such as obfuscation or fragmentation, to bypass WAFs and deliver their payloads. Additionally, WAFs may not be able to detect attacks that are embedded within legitimate traffic.

3.2 Proposed System

In WAF involves the use of advanced technologies and techniques to enhance the security of web applications. One approach is to incorporate machine learning algorithms that can detect and respond to new and emerging threats in real-time. This can enable the WAF to learn from past attack patterns and adjust its rules accordingly, improving its accuracy and effectiveness. Another approach is to use behavioral analysis to identify abnormal traffic patterns and potential attacks. By monitoring traffic over time and analyzing user behavior, a WAF can detect attacks that would not be identified by simple signature-based methods. The proposed system may

also use cloud-based WAF solutions that leverage the power of cloud computing to provide scalable, high-performance protection against a range of web application threats.

One major advantage of a proposed WAF system is improved accuracy and effectiveness in identifying and blocking potential threats. By leveraging advanced technologies and techniques, the proposed system can better protect web applications from a wide range of attacks and vulnerabilities. Additionally, the proposed system can reduce the complexity of setting up and maintaining a WAF. By automating rule generation and tuning, the proposed system can help to minimize false positives and false negatives, reducing the burden on security teams.

3.3 Feasibility Study

3.3.1 Economic Feasibility

The implementing a WAF should be evaluated in relation to the specific project and organization considering its implementation. The project's goals, budget, and scalability should be considered when assessing the economic feasibility of the WAF. A cost-benefit analysis should be conducted to determine the cost of implementing and maintaining the WAF against the potential benefits of increased security and reduced risk of data breaches and attacks. The organization's growth plans should also be taken into account, and the WAF's scalability should be assessed to ensure it can meet the increased demand if the organization expects to expand its web applications in the future. The economic feasibility of a WAF should be evaluated based on the project's specific needs and goals, taking into account factors such as cost, budget, scalability, and potential benefits.

Overall, the economic feasibility of a WAF should be evaluated based on the specific needs and goals of the project and organization, taking into account factors such as cost, budget, scalability, and potential benefits.

3.3.2 Technical Feasibility

The WAF should be evaluated in the context of the specific project and organization that is considering its implementation. The following factors should be considered:
Compatibility: The WAF should be compatible with the existing technology stack, including the web server, application server, and database server.

Performance: The WAF should be able to handle the expected web traffic and requests without causing significant performance degradation. The WAF's impact on application performance should also be evaluated.
Integration: The WAF should be integrated seamlessly into the existing infrastructure and workflows, without disrupting other systems or requiring significant changes to existing processes.
Maintenance: The WAF should be easy to maintain and update, with minimal disruption to ongoing operations.
Customization: The WAF should be customizable to meet the specific needs of the project and organization, including the ability to create custom rules and policies.

3.3.3 Social Feasibility

The social feasibility of implementing a WAF should be evaluated in the context of the specific project and organization considering its implementation. The following factors should be considered:
User Acceptance: The WAF should not negatively impact the user experience of the web application. Any changes to the application's functionality or user interface should be minimized to ensure user acceptance.
Training: If the WAF requires specialized knowledge or training to operate, the organization should ensure that staff members have the necessary training to effectively use the WAF.
Compliance: The WAF should comply with any applicable legal or regulatory requirements, including data privacy laws and industry-specific regulations.
Perception: The implementation of the WAF should be communicated effectively to stakeholders to ensure that it is perceived as a positive step toward enhancing security and protecting sensitive data.

3.4 System Specification

3.4.1 Hardware Specification

Processor: Intel Xeon or AMD EPYC processors, with multiple cores and high clock speeds.

Memory: At least 16 GB of RAM, with 32 GB or more recommended for high-traffic environments.
Storage: SSD with at least 500 GB of storage capacity.
Network Interfaces: Multiple Gigabit Ethernet or 10 Gigabit Ethernet interfaces, with support for load balancing and failover.
Encryption Acceleration: Hardware-based encryption acceleration, such as Intel QuickAssist Technology, to offload encryption processing from the CPU.
Redundancy: Redundant power supplies and fans for high availability.
Form Factor: Rack-mountable form factor, with 1U or 2U height for space-saving purposes.
These hardware specifications can vary based on the specific needs of the project and organization. It's important to consult with a vendor or expert to determine the appropriate hardware specifications for your web application firewall.

3.4.2 Software Specification

Operating System: Linux-based operating system, such as CentOS, Ubuntu, or Debian.
Web Server Support: Support for popular web servers, such as Apache, Nginx, and Microsoft IIS.
Configuration Management: Configuration management tools, such as Ansible or Puppet, to automate configuration and deployment of the web application firewall.
Rule Management: User-friendly rule management interfaces, such as web-based GUIs, for easy management of security policies and rules.
Real-time Monitoring: Real-time monitoring and logging capabilities to provide visibility into web traffic and potential security threats.
Integration with SIEM: Integration with SIEM systems to provide centralized management and analysis of security events.
API Integration: Support for API integration, allowing developers to integrate the web application firewall with other applications and services.
These software specifications can vary based on the specific needs of the project and organization. It's important to consult with a vendor or expert to determine the appropriate software specifications for your web application firewall.

3.4.3 Standards and Policies

WAFs should adhere to industry standards and policies to ensure the highest level of security for web applications. Here are some of the most important standards and policies related to WAFs:

OWASP Top 10: The OWASP Top 10 is a list of the most critical security risks to web applications. A WAF should be designed to protect against these risks, including SQL injection, cross-site scripting , and broken authentication and session management.

PCI DSS: The PCI DSS is a set of security standards designed to ensure that companies that accept, process, store or transmit credit card information maintain a secure environment. A WAF can be a critical component of compliance with PCI DSS.

GDPR: The GDPR is a European Union regulation designed to protect the privacy and data of EU citizens. A WAF can help organizations comply with GDPR by protecting personal data and preventing data breaches.

HIPAA: The HIPAA is a U.S. law that requires healthcare organizations to protect the confidentiality, integrity, and availability of patients' protected health information (PHI). A WAF can help healthcare organizations comply with HIPAA by protecting patient data from web application attacks.

NIST Cybersecurity Framework: The National Institute of Standards and Technology (NIST) Cybersecurity Framework provides guidelines for improving cybersecurity for critical infrastructure. A WAF can help organizations implement controls related to access control, threat detection, and response.

Overall, a WAF should be designed to adhere to industry standards and policies related to web application security, data privacy, and cybersecurity.

Chapter 4

METHODOLOGY

4.1 Reverse Proxy Server

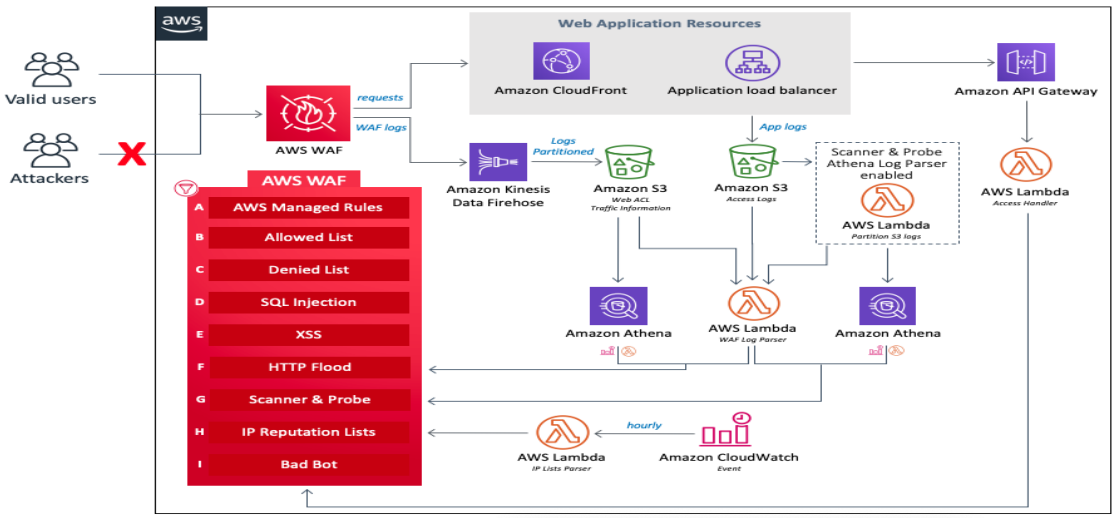


Figure 4.1: **Reverse Proxy Server**

Reverse Proxy Server: In figure 4.1 the reverse proxy server sits in front of the web server and intercepts incoming requests from clients. The reverse proxy server can filter and analyze traffic before forwarding it to the web server. Rule Engine: The rule engine is the core of the WAF and is responsible for inspecting traffic and enforcing security policies. The rule engine can be configured to detect and block malicious traffic based on predefined rules and policies. Logging and Monitoring: The WAF should have logging and monitoring capabilities to track traffic and security events. This can include real-time monitoring, alerting, and reporting capabilities to provide visibility into potential threats.Integration: The WAF can integrate with other security tools and systems, such as SIEM systems, to provide a comprehensive security posture.Management Interface: The WAF should have a user-friendly management interface that allows security administrators to configure, monitor, and manage the security policies and rules.Load Balancing: The WAF can also provide load balancing capabilities to distribute traffic across multiple web servers, improving the

performance and availability of the web application.

4.2 Design Phase

4.2.1 Data Flow Diagram

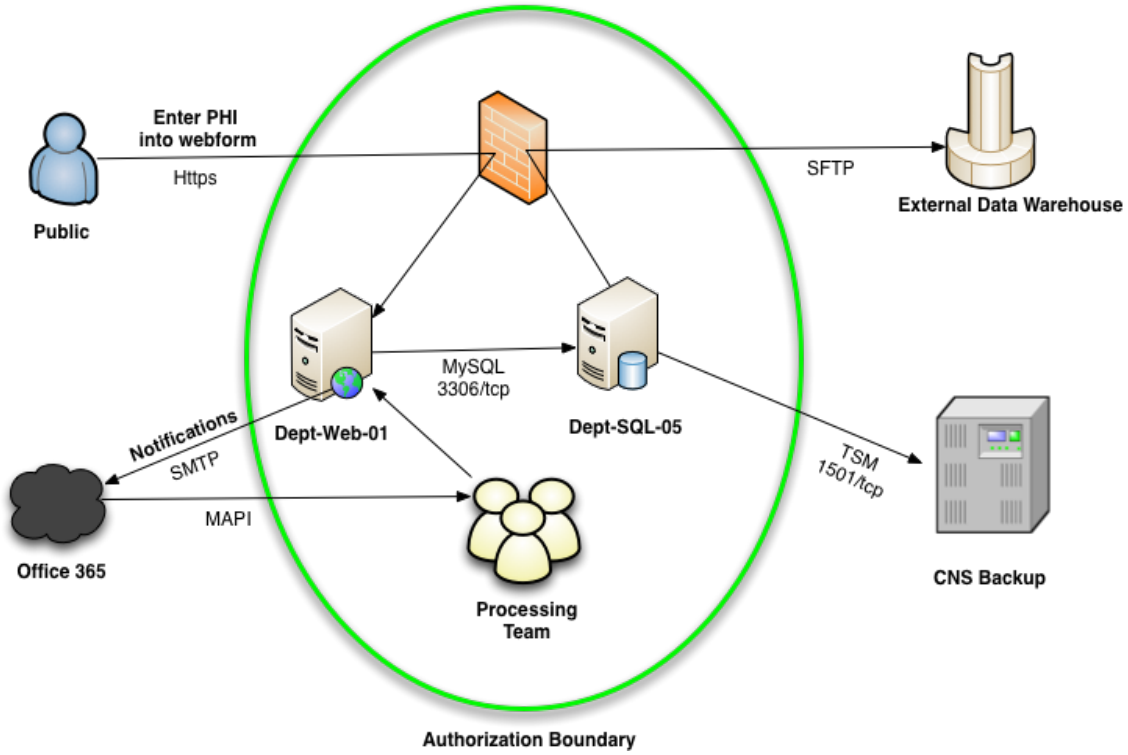


Figure 4.2: **Data Flow Diagram**

Level 0:The highest-level in the above figure 4.2 for a web application firewall consists of one process, which represents the entire system.
Level 1:The Level 1 in the above figure 4.2 breaks down the process from the Level 0 diagram into three main components:Web Server,WAF,Client.
The Level 2: In the above figure 4.2 provides more detail about the data flows between the components of the system: Web sever, WAF, Client.

4.2.2 Use Case Diagram

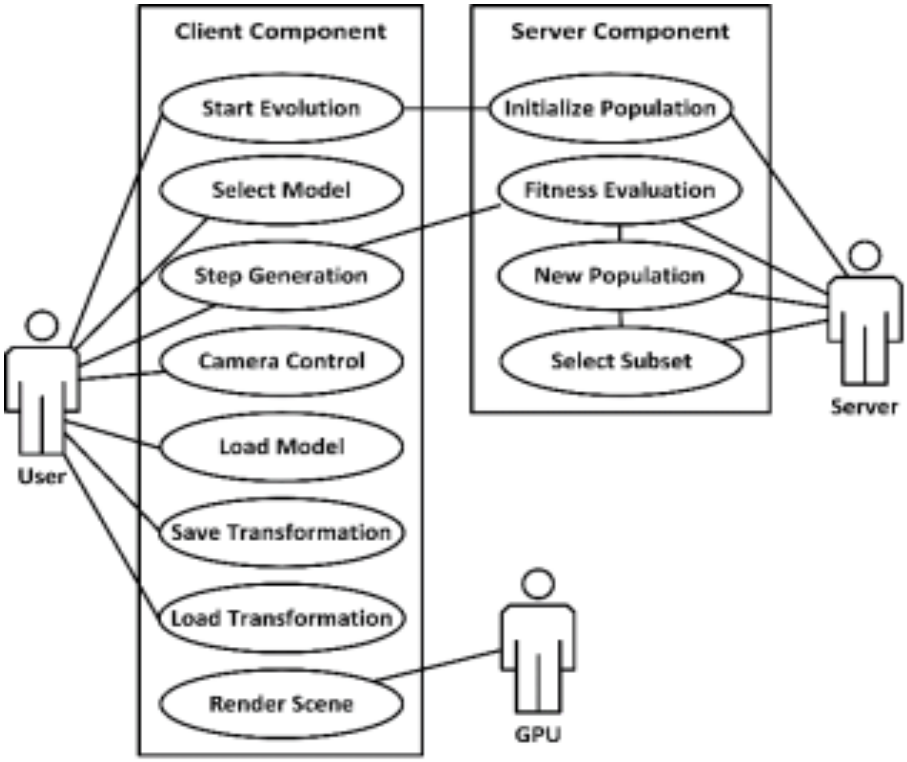


Figure 4.3: **Use Case Diagram**

In the use case diagram figure 4.3 for a WAF shows the various actors and their interactions with the system. Here is a description of the main actors and use cases: Configure WAF, Monitor WAF, Manage WAF. Security Analyst, Web Server, Receive Requests, Send Requests.

4.2.3 Class Diagram

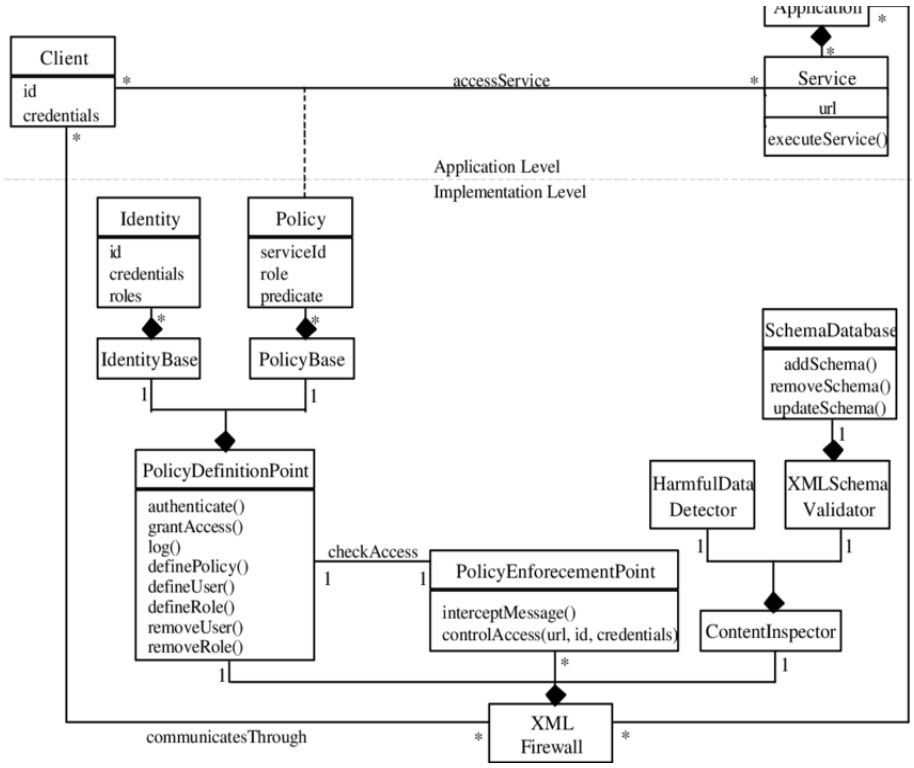


Figure 4.4: **Class Diagram**

In class diagram of figure 4.4 for a WAF shows the classes and their relationships within the system. Here is a description of the main classes and their attributes and methods: WAF, Rule Engine, Security Policy, Traffic, Log. Overall, the class diagram for a WAF illustrates the different classes and their relationships within the system. The WAF class is the main class, which interacts with the rule engine, security policies, traffic, and logs to provide web application security. Overall, the class diagram for a WAF illustrates the different classes and their relationships within the system. The WAF class is the main class, which interacts with the rule engine, security policies, traffic, and logs to provide web application security.

4.2.4 Sequence Diagram

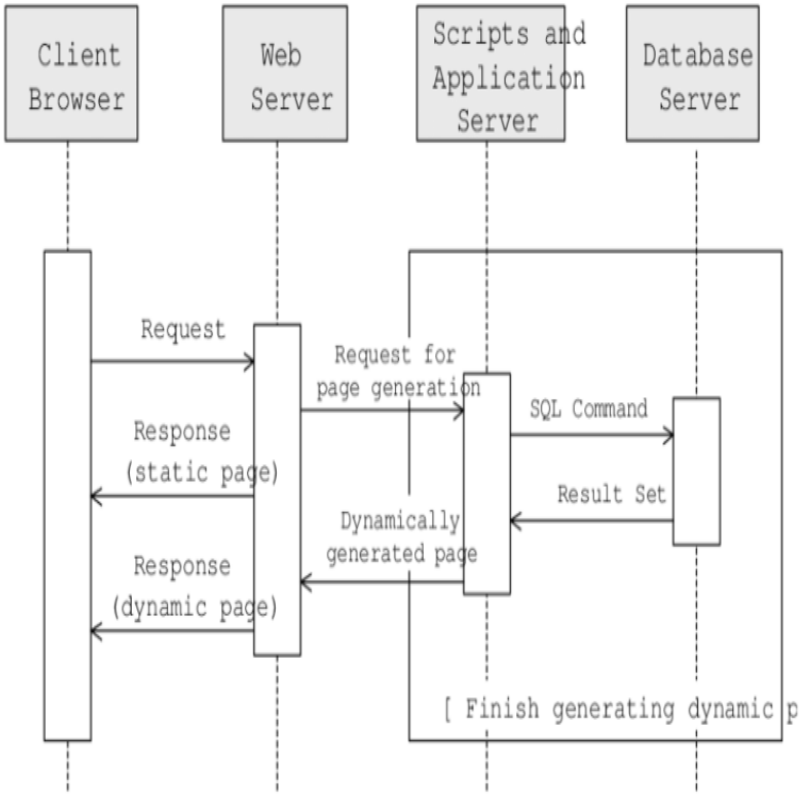


Figure 4.5: **Sequence Diagram**

In this figure 4.5 the user sends an HTTP request to a web application. The web application then sends the request to the WAF for inspection. The WAF inspects the request to determine if it's legitimate or an attack. If it's an attack, the WAF blocks the request and notifies the user. If it's legitimate, the WAF allows the request to continue to the web application.After the web application processes the request, it sends an HTTP response back to the user. The WAF inspects the response to determine if it's legitimate or an attack. If it's an attack, the WAF blocks the response and notifies the user. If it's legitimate, the WAF allows the response to continue to the user.

4.2.5 Collaboration Diagram

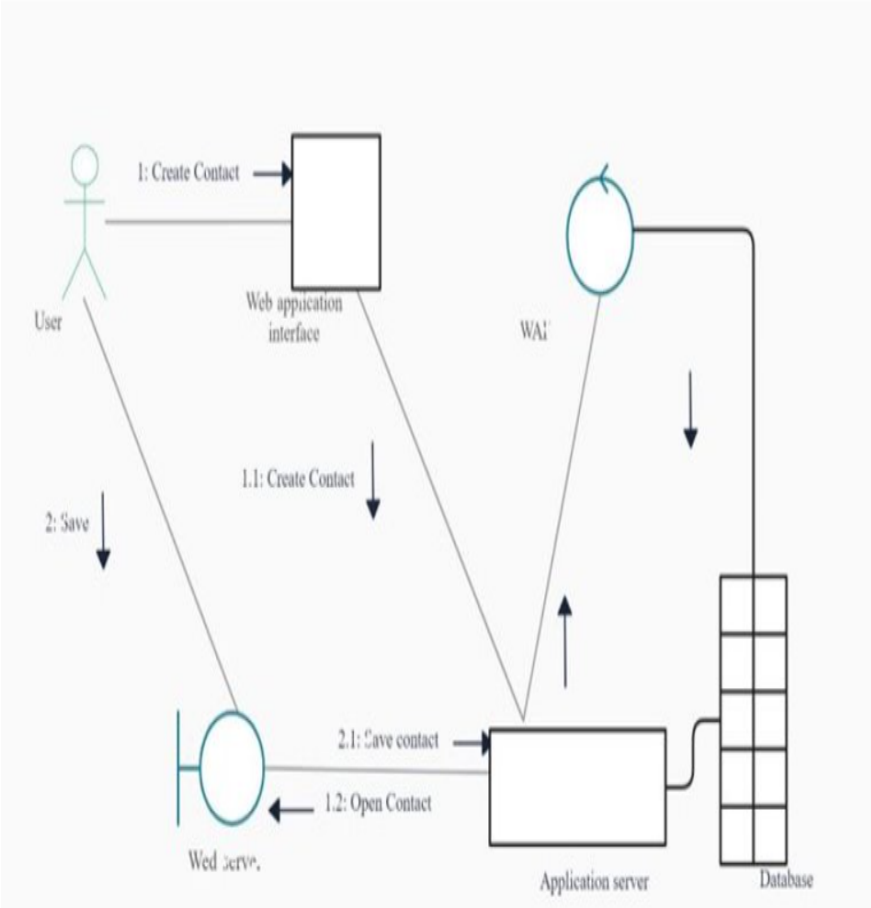


Figure 4.6: **Collaboration Diagram**

In this figure 4.6, the user interacts with the web application through an HTTP request. The web application receives the request and forwards it to the WAF for inspection.The WAF inspects the request and determines whether it's legitimate or an attack. If the request is an attack, the WAF blocks it and notifies the user. If the request is legitimate, the WAF allows it to pass through to the web application.Once the web application has processed the request, it sends an HTTP response back to the WAF for inspection. The WAF inspects the response and determines whether it's legitimate or an attack. If the response is an attack, the WAF blocks it and notifies the user. If the response is legitimate, the WAF allows it to pass through to the user.

4.2.6 Activity Diagram

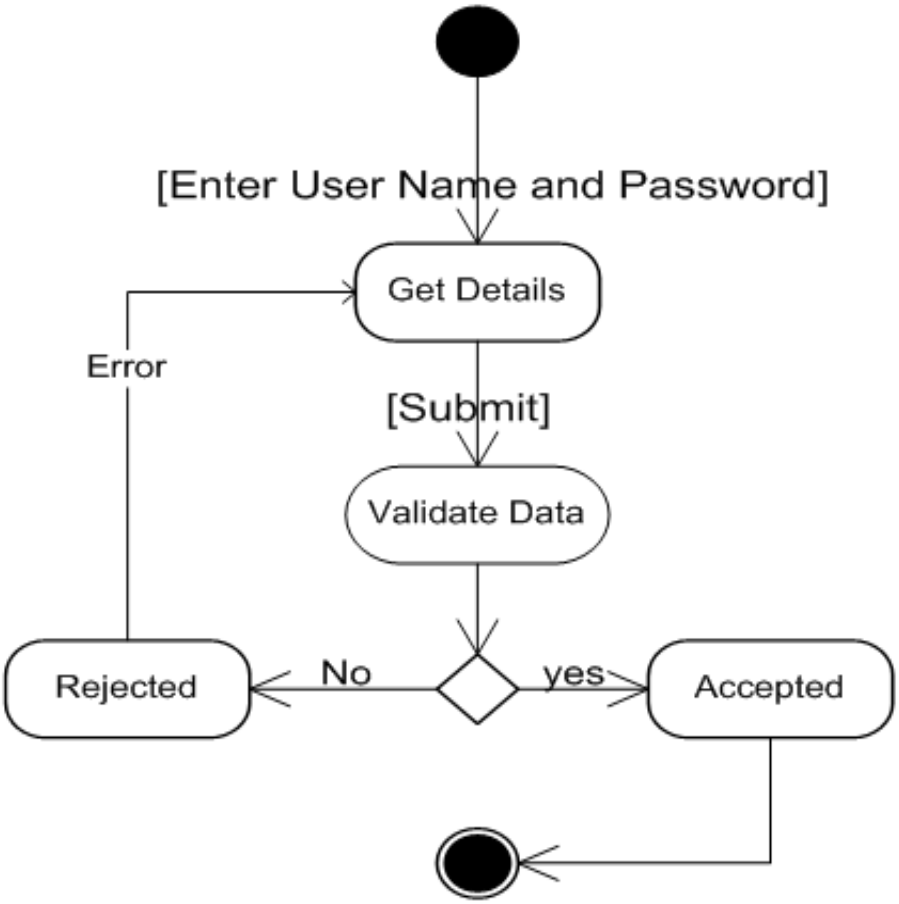


Figure 4.7: **Activity Diagram**

In this figure 4.7 **Incoming request**: The web application firewall receives an incoming request from a user.
Check Request: The web application firewall checks the request against its predefined security policies and rules.
Block Request: If the request violates any of the security policies or rules, the web application firewall will block the request and proceed to the logging and notification steps.
Log Request: The web application firewall logs the details of the blocked request for future reference and analysis.
Notify Security Team: The web application firewall notifies the security team about the blocked request, so that they can take appropriate actions to mitigate the security risk.
Allow Request: If the request passes all the security checks, the web application firewall allows the request to reach its destination.

4.3 Algorithm & Pseudo Code

4.3.1 Algorithm

1.The WAF inspects the incoming HTTP request, including the URL, headers, and payload.
2.The WAF compares the request to a set of predefined rules or policies that define what types of requests are allowed or blocked.
3.If the request violates one or more of the rules, the WAF takes action to block the request, such as dropping the connection or returning an error message to the client.
4.If the request passes all of the rules, the WAF allows it to proceed to the web application.
5.The WAF may also monitor outgoing traffic from the web application to detect and block any attempts to exfiltrate data or conduct other malicious activities.
6.The WAF may also maintain logs of incoming and outgoing traffic for auditing and analysis purposes.

4.3.2 Pseudo Code

```
// Define the firewall's security policies and rules
   var policies =
   "Disallow SQL injection": ["SELECT", "INSERT", "UPDATE", "DELETE"],
   "Disallow cross-site scripting (XSS)": ["¡script¿", "onmouseover", "onload"],
   "Disallow directory traversal": ["../", "/..", "..
"],
   // ... add more policies and rules as needed
   ;
   // Define a function to check the request against the security policies and rules
   function checkRequest(request)
   // Loop through all the policies
   for (var policy in policies)
   // Loop through all the rules in the current policy
   for (var i = 0; i ¡ policies[policy].length; i++)
```

```
// If the current rule is found in the request, block the request.
if (request.contains(policies[policy][i]))
logRequest(request); // Log the blocked request
notifySecurityTeam(request); // Notify the security team
return "BLOCKED";
// If none of the policies and rules are violated, allow the request return "ALLOWED";
// Define a function to log the details of the blocked request function logRequest(request)
// Log the request details to a file or database
// ...
// Define a function to notify the security team about the blocked request function notifySecurityTeam(request)
// Send an email or text message to the security team
// ...
// Test the firewall with a sample request
var request = "SELECT * FROM users WHERE username = 'admin';";
var result = checkRequest(request);
if (result == "BLOCKED")
// Do something to handle the blocked request, such as displaying an error message
else
// Proceed with the request as normal
// ...
```

4.4 Module Description

4.4.1 Traffic Interception Module

The Traffic Interception Module is a key component of a WAF. It is responsible for intercepting incoming traffic before it reaches the web application server and analyzing it for potential security threats.

Here are some of the key features and functions of the Traffic Interception Module in a WAF:

Protocol Support: The Traffic Interception Module supports different protocols such as HTTP, HTTPS, SMTP, FTP, and others. It can analyze traffic at the ap-

plication layer and block requests that violate security policies.
Request Analysis: The module analyzes incoming requests to identify malicious traffic. It inspects the request headers and payload and compares them against predefined rules or machine learning models.
Payload Inspection: The Traffic Interception Module can analyze the payload of incoming requests to identify potential attacks like SQL injection, cross-site scripting, or file inclusion attacks.
Request Blocking: The module can block requests that violate predefined rules or are deemed malicious. It can return a 403 Forbidden response or redirect the user to a safe page.
Content Filtering: The Traffic Interception Module can filter content and block requests that contain specific content like malware, viruses, or inappropriate content.
Traffic Shaping: The module can limit the number of requests or bandwidth of incomingtraffic to prevent Denial of Service (DoS) or Distributed Denial of Service (DDoS) attacks.
SSL/TLS Inspection: The Traffic Interception Module can decrypt and inspect SSL/TLS encrypted traffic to identify potential threats. It can also re-encrypt the traffic before forwarding it to the web application server.

4.4.2 Rule Engine Module

The Rule Engine Module is a critical component of a WAF. It is responsible for analyzing incoming traffic and comparing it against predefined security rules to determine whether it is malicious or not. The Rule Engine Module typically consists of a set of predefined security rules that define what types of traffic are allowed and what types are blocked.
Here are some key features and functions of the Rule Engine Module in a WAF:
Predefined Security Rules: The Rule Engine Module contains a set of predefined security rules that define what types of traffic are allowed and what types are blocked. These rules are designed to protect against common attacks like SQL injection, cross-site scripting, and file inclusion.
Real-time Rule Updates: The Rule Engine Module can be updated in real-time to adapt to changing security threats. New security rules can be added, and existing rules can be modified to provide better protection against new attack vectors.

Flexible Rule Engine: The Rule Engine Module is typically designed to be flexible and customizable. It allows the user to define their own custom rules to match their specific security needs.
Block or Allow Traffic: Based on the predefined rules, the Rule Engine Module determines whether incoming traffic should be allowed or blocked. If the traffic violates any of the rules, it is blocked, and the user is either redirected to a safe page or denied access to the web application.
Machine Learning-Based Rules: Some Rule Engine Modules may use machine learning algorithms to learn the normal behavior of the web application and identify anomalous behavior. Machine learning-based rules can improve the accuracy of the WAF and reduce false positives.
Event Logging: The Rule Engine Module maintains logs of all incoming traffic and whether it was allowed or blocked. These logs can be used for analysis and auditing purposes.

4.4.3 Logging and Reporting Module

The Logging and Reporting Module is a crucial component of a WAF. It is responsible for logging all incoming traffic and generating reports that provide insights into web application security. The Logging and Reporting Module typically consists of a logging mechanism that captures traffic data and a reporting engine that processes and presents the data in a meaningful way.
Here are some key features and functions of the Logging and Reporting Module in a WAF:
Traffic Logging: The Logging and Reporting Module captures traffic data such as request method, request URL, client IP address, user agent, and response status code. It logs all incoming traffic, including allowed and blocked traffic.
Detailed Reports: The Reporting Engine processes the logged data and generates detailed reports that provide insights into web application security. Reports can include information such as the number of requests, the types of requests, the most frequently blocked requests, and the top sources of attacks.
Alerting: The Logging and Reporting Module can generate alerts when certain events occur, such as when an attack is detected or when a predefined threshold is exceeded.Customizable Reports: The Reporting Engine is typically designed to

be customizable. It allows the user to define their own reports and specify which data to include in the reports.

Historical Data: The Logging and Reporting Module stores historical data that can be used for analysis and auditing purposes. The user can view trends over time, identify patterns of attack, and determine whether the WAF is effectively protecting the web application.

Integration with SIEM: The Logging and Reporting Module can integrate with SIEM systems to provide a centralized view of security events across the organization.

4.5 Steps to execute/run/implement the Project

4.5.1 Data Preparation

Data preparation for a WAF involves several steps to ensure that the firewall is able to effectively protect web applications from various types of attacks.

4.5.2 Model Training

Model training is a crucial step in developing a machine learning-based WAF that can accurately detect and block malicious traffic.

4.5.3 Vulnerability Detection

Vulnerability detection is a critical aspect of a WAF functionality, as it helps to identify and prevent potential security weaknesses in web applications.

Chapter 5

IMPLEMENTATION AND TESTING

5.1 Input and Output

5.1.1 Input Design

```
UnitTest1.cs   Test Explorer   UnitTest1.cs   Program.cs
HelloWorld.Tests                                HelloWorld.T
1   using NUnit.Framework;
2   using System.IO;
3   using System;
4
5   namespace HelloWorld.Tests
6   {
        0 references
7       public class Tests
8       {
9           private const string Expected = "Hello World!";
10          [SetUp]
            0 references
11          public void Setup()
12          {
13          }
14
15          [Test]
            | 0 references
16          public void Test1()
17          {
18              // Assert.Pass();
19              using (var sw = new StringWriter())
20              {
21                  Console.SetOut(sw);
22                  HelloWorld.Program.Main();
23
24                  var result = sw.ToString().Trim();
25                  Assert.AreEqual(Expected, result);
26              }
27          }
28      }
29  }
```

Figure 5.1: **Input Design**

In the figure 5.1 the input design of a WAF is an important factor in its effectiveness. The input design determines how the WAF processes and filters the incoming traffic to the web application, which can have a significant impact on the WAF's ability to detect and prevent web application attacks.

5.1.2 Output Design

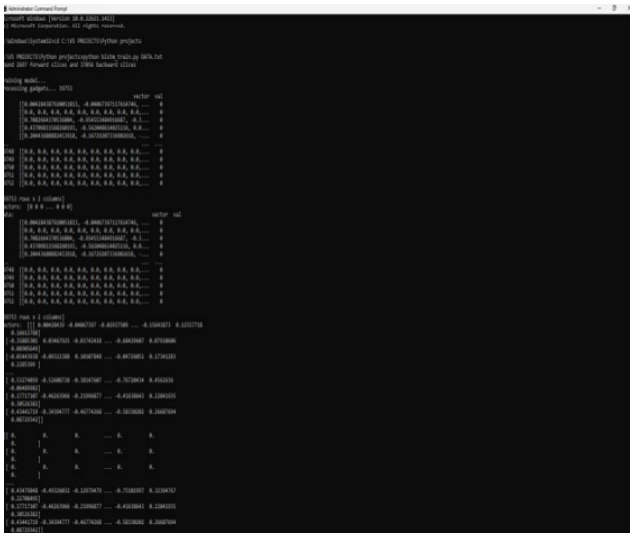


Figure 5.2: **Output Design**

In the figure 5.2 the output design of a WAF is an important factor in its effectiveness. The output design of a WAF determines how the WAF reports its actions and alerts to the users and administrators, which can impact the usability, visibility, and manageability of the WAF.

5.2 Testing

5.3 Types of Testing

5.3.1 Unit Testing

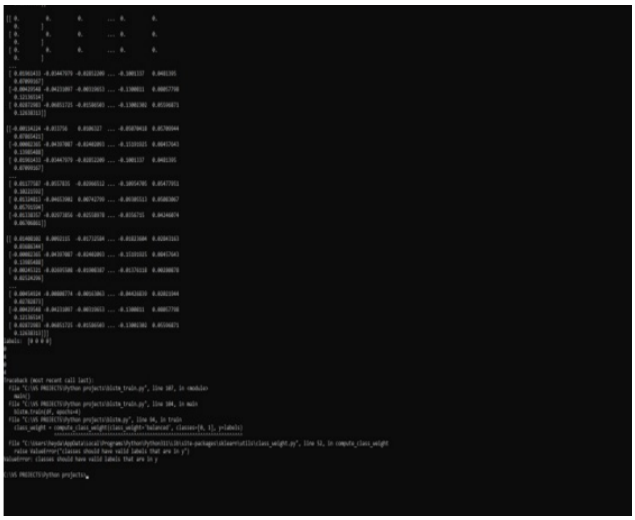


Figure 5.3: **Unit Testing**

In this figure 5.3 unit testing for automated vulnerability detection using deep learning techniques is an important step to ensure that the system is working correctly and detecting vulnerabilities accurately. Unit testing can help uncover defects and issues early in the development process. By running automated unit tests, developers can catch errors before they become more complicated and expensive to fix. With the proposed system, unit testing can verify the correctness of the models and ensure their accuracy, which can lead to a higher detection rate and fewer false positives and false negatives.

5.3.2 Integration Testing

Input

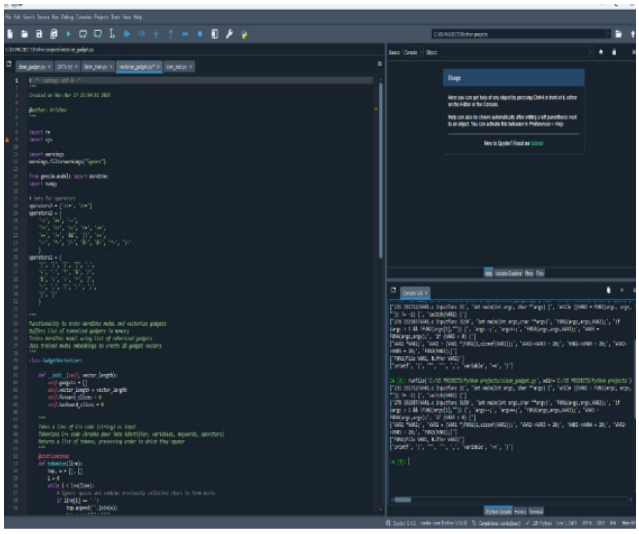


Figure 5.4: **Integration Testing**

In this figure 5.4 integration testing is a critical step in ensuring the effectiveness of a web application firewall (WAF) in detecting and preventing attacks. Integration testing involves testing the WAF's integration with the web application to ensure that it works as intended and does not interfere with the application's functionality.

5.3.3 System Testing

Input

```
Epoch 1/5
3452/3452 [==============================] - 3s 727us/step - loss: 6907936.0000 - accuracy: 0.9472
Epoch 2/5
3452/3452 [==============================] - 2s 663us/step - loss: 6835772.0000 - accuracy: 0.9533
Epoch 3/5
3452/3452 [==============================] - 2s 675us/step - loss: 3928674.5000 - accuracy: 0.9470
Epoch 4/5
3452/3452 [==============================] - 2s 677us/step - loss: 4201450.5000 - accuracy: 0.9471
Epoch 5/5
3452/3452 [==============================] - 2s 675us/step - loss: 2288503.0000 - accuracy: 0.9462

Out[58]: <tensorflow.python.keras.callbacks.History at 0x20c96bbe7c8>
```

Figure 5.5: **System Testing**

In this figure 5.5 system testing is an essential step in ensuring the overall effectiveness and reliability of a WAF in protecting against web application attacks. System testing involves testing the WAF as a whole system to ensure that it works as intended and meets the necessary security requirements.

5.3.4 Test Result

```
albert@BSD13:~ % wafw00f http://172.23.69.232

              ____
             /    \
            ( W00f! )
             \ ___/
                ,,    __
           |`-.__   / /
           /"  _/  /_/
          *===*    /
         /     )__//
   /|   /     /---`
   \\/`   \ |
   `\    /_\\_
     `     ``-`
     ____

                404 Hack Not Found
                        __    __
                        \ \   / /
                         \ \_/ /  405 Not Allowed
                          \   /
                     403 Forbidden
                          / _ \
        502 Bad Gateway  / / \ \  500 Internal Error
                        /_/   \_\

                    ~ WAFW00F : v2.1.0 ~
       The Web Application Firewall Fingerprinting Toolkit

[*] Checking http://172.23.69.232
[+] Generic Detection results:
[-] No WAF detected by the generic detection
[~] Number of requests: 7
albert@BSD13:~ %
```

Figure 5.6: **Result**

Chapter 6

RESULTS AND DISCUSSIONS

6.1 Efficiency of the Proposed System

The efficiency of a proposed system of WAF can depend on several factors, including the type of attacks the WAF is designed to detect, the configuration of the WAF, and the resources available for the WAF to operate. When a WAF is properly configured and deployed, it can be very efficient at detecting and blocking attacks on a web application. However, a poorly configured or incorrectly deployed WAF may generate false positives or fail to detect attacks, which can reduce its efficiency. Another factor that can affect the efficiency of a WAF is the resources available for it to operate. A WAF that is under-resourced may not be able to keep up with the volume of traffic on a web application, leading to delays or failures in detecting attacks. On the other hand, a WAF that is over-resourced may waste resources and increase costs unnecessarily.

The effectiveness of a WAF can also be impacted by the type of attacks it is designed to detect. For example, signature-based WAFs are very efficient at detecting known attacks, but may not be as effective at detecting new or unknown attacks. In contrast, behavior-based WAFs may be more effective at detecting new or unknown attacks, but may be more resource-intensive to operate.Overall, the efficiency of a proposed system of web application firewall will depend on several factors, including its configuration, deployment, and resources available, as well as the type of attacks it is designed to detect. A properly configured and deployed WAF can be an efficient and effective tool for protecting a web application from a variety of attacks.

6.2 Comparison of Existing and Proposed System

Existing system:
The existing system of WAF may involve several matters, such as:
Outdated Technology: The existing WAF system may use outdated technology that

cannot effectively protect the web application against the latest threats. This may make the WAF less efficient in detecting and mitigating attacks, leaving the web application vulnerable to security breaches.

Lack of Scalability: The existing WAF system may be unable to scale up or down to accommodate changes in web application traffic. This can lead to performance issues or downtime during peak traffic periods, affecting the user experience.

False Positives: The existing WAF system may generate false positives, which are alerts triggered by legitimate traffic that is misidentified as malicious. False positives can be disruptive to web application users and make it difficult for security teams to distinguish legitimate traffic from malicious traffic.

Difficult to Manage: The existing WAF system may be difficult to manage and configure, leading to errors or misconfigurations that can leave the web application vulnerable. This can also make it challenging for security teams to stay up-to-date with the latest threats and apply patches or updates to the WAF.

High Cost of Ownership: The existing WAF system may be expensive to maintain or require significant investment in hardware or software licenses. This can make it challenging for smaller organizations or those with limited resources to implement an effective WAF..

Proposed system:(Random forest algorithm)

The proposed system of WAF may involve several matters, such as:

Advanced Technology: The proposed WAF system may use advanced technology that can better protect the web application against the latest threats. This can include machine learning algorithms, behavior-based detection mechanisms, or other advanced techniques that can more effectively detect and mitigate attacks.

Scalability: The proposed WAF system may be designed to scale up or down to accommodate changes in web application traffic. This can ensure that the web application remains available and responsive, even during peak traffic periods.

Reduced False Positives: The proposed WAF system may be optimized to reduce false positives, which are alerts triggered by legitimate traffic that is misidentified as malicious. This can reduce disruption to web application users and improve the accuracy of the WAF.

User-Friendly Interface: The proposed WAF system may have a user-friendly interface that simplifies management and configuration. This can reduce the risk of errors or misconfigurations that can leave the web application vulnerable.

Cost-Effectiveness: The proposed WAF system may offer cost-effective alternatives

such as cloud-based solutions or open-source software that can reduce the overall cost of ownership. This can make it easier for smaller organizations or those with limited resources to implement an effective WAF.

6.3 Sample Code

```
1. while (incoming_request)
2.     request = incoming_request()
3.     if (request_violates_rules(request))
4.         block_request()
5.     else
6.         forward_request()
7.     end if
8. end while

9. function request_violates_rules(request)
10.     for each rule in ruleset
11.         if (rule.matches(request))
12.             return true
13.         end if
14.     end for
15.     return false
16. end function

17. function block_request()
18.     send_response("Access denied", 403)
19.     end
20. end function

21. function forward_request()
22.     send_request(request)
23.     receive_response()
24.     send_response(response)
25.     end
26. end function

main code
// Import necessary libraries
import java.net.*;
import java.io.*;
import java.util.*;

// Define main class for the WAF
public class WebAppFirewall {

    // Define main method to intercept and analyze HTTP traffic
    public static void main(String[] args) throws IOException {
```

```java

        // Create a socket to listen for incoming HTTP requests on port 80
        ServerSocket serverSocket = new ServerSocket(80);

        // Create an infinite loop to continuously process incoming requests
        while (true) {

            // Accept incoming HTTP request and establish a connection
            Socket clientSocket = serverSocket.accept();

            // Create input and output streams to read and write HTTP request and response data
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream
                ()));
            DataOutputStream out = new DataOutputStream(clientSocket.getOutputStream());

            // Read HTTP request data
            String requestLine = in.readLine();
            String headers = "";
            String line = in.readLine();
            while (line != null && line.length() > 0) {
                headers += line + "\n";
                line = in.readLine();
            }

            // Analyze HTTP request data to determine whether to allow or block traffic
            if (isMalicious(requestLine, headers)) {
                // Block traffic and return a 403 Forbidden response
                out.writeBytes("HTTP/1.1 403 Forbidden\r\n\r\n");
            } else {
                // Allow traffic and forward the HTTP request to the web server
                Socket webServerSocket = new Socket("localhost", 8080);
                DataOutputStream webOut = new DataOutputStream(webServerSocket.getOutputStream());
                BufferedReader webIn = new BufferedReader(new InputStreamReader(webServerSocket.
                    getInputStream()));
                webOut.writeBytes(requestLine + "\r\n" + headers + "\r\n");
                String response = "";
                line = webIn.readLine();
                while (line != null) {
                    response += line + "\n";
                    line = webIn.readLine();
                }
                out.writeBytes(response);
                webOut.close();
                webIn.close();
                webServerSocket.close();
            }

            // Log HTTP request and response data for future analysis and reporting
            log(requestLine, headers);

```

```
            // Close input and output streams and client socket connection
            in.close();
            out.close();
            clientSocket.close();
        }
    }

    // Define method to analyze HTTP request data and determine whether to allow or block traffic
    private static boolean isMalicious(String requestLine, String headers) {
        // Apply rules and policies to analyze HTTP request data and identify malicious traffic
        // Return true if traffic is malicious, false if it is benign
    }

    // Define method to log HTTP request and response data
    private static void log(String requestLine, String headers) {
        // Write log data to file or database for future analysis and reporting
    }
}
```

Output

```
└─$ wafw00f https://example.com

            ______
           /      \
          (  W00f! )
           \  ____/
                   ,,    __           404 Hack Not Found
            |`-.__   / /
            /"  _/  /_/                     __     __
           *===*    /                       \\    //
          /     )__//                        \ \_/ /   405 Not Allowed
     /|  /     /---`                          \    /
     \\/`   \ |                            403 Forbidden
     `\    /_\\_                                / _ \
       `     ``-`            502 Bad Gateway   / / \ \   500 Internal Error
      _____                                    /_/   \_\

                    ~ WAFW00F : v2.1.0 ~
       The Web Application Firewall Fingerprinting Toolkit

[*] Checking https://example.com
[+] The site https://example.com is behind Edgecast (Verizon Digital Media) WAF.
[~] Number of requests: 2
```

Figure 6.1: **WAF**

In the figure 6.1 a WAF is a type of firewall that is designed to protect web application from attacks and unauthorized access. It is a security that sits between a web application and the internet, analyzing incoming traffic and blocking malicious while allowing legitimate to pass through.

```
root@clueless:~# wafw00f http://ethicalhackingblog.com

                                ^    ^

      _  __  _   ___ _  __ _   _  ___
     ///7/ /.' \ / _////7/ /,' \ ,' \ / _/
    | V V // o // _/ | V V // 0 // 0 // _/
    |_n_,'/_n_//_/   |_n_,' \_,' \_,'/_/
                                <
                                 ...'

    WAFW00F - Web Application Firewall Detection Tool

    By Sandro Gauci && Wendel G. Henrique

Checking http://ethicalhackingblog.com
Generic Detection results:
The site http://ethicalhackingblog.com seems to be behind a WAF or some sort of
security solution
Reason: Blocking is being done at connection/packet level.
Number of requests: 12
root@clueless:~#
```

Figure 6.2: **WAF Detection Tools**

In figure 6.2, this is a Python-based tool that can detect the presence of a WAF by analyzing the HTTP response headers and looking for specific WAF signatures. Website is protected by WAF. These tools are typically used by security researchers, penetration testers and other security professionals to assess the security of web applications and identify vulnerabilities that could be exploited by attackers.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

WAF is an essential security tool that helps protect web applications from a wide range of cyber threats. It acts as a filter between the web application and the internet, identifying and blocking malicious traffic while allowing legitimate traffic to pass through. An effective WAF system can offer advanced technology such as machine learning algorithms or behavior-based detection mechanisms and it can be optimized to reduce false positives and be easy to manage. It can also scale up or down to accommodate changes in web application traffic and be cost-effective, making it accessible to a wide range of organizations. However, the effectiveness of a WAF system is dependent on its ability to keep up with the latest threats and trends in cyber attacks. It is essential to regularly update and maintain the WAF system to ensure that it remains effective in protecting the web application. Overall, a well-designed and properly maintained WAF system can provide a critical layer of security for web applications, helping to prevent data breaches, unauthorized access, and other cyber threats.

7.2 Future Enhancements

The field of web application security is constantly evolving, and there are several potential future enhancements for WAF technology that can help improve its effectiveness. One possible future enhancement is the use of artificial intelligence and machine learning algorithms to better detect and prevent sophisticated attacks. These advanced techniques can help WAF systems more effectively identify and respond to threats, reducing the risk of data breaches or other security incidents.

Another potential enhancement is the integration of WAF technology with other security solutions, such as IDS or SIEM systems. This can help provide a more comprehensive view of the security posture of the web application and allow for faster and more effective incident response.Additionally, the future of WAF technology may involve more cloud-based solutions that can offer greater scalability and flexibility, as well as improved threat intelligence capabilities. This can help organizations better protect their web applications against a wide range of cyber threats.

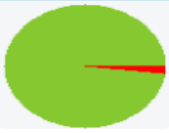
Chapter 8

PLAGIARISM REPORT

Check Plagiarism

PLAGIARISM SCAN REPORT

Date	May 08, 2023
Exclude URL:	NO



Unique Content	98
Plagiarized Content	2

Word Count	971
Records Found	0

CONTENT CHECKED FOR PLAGIARISM:

A Web Application Firewall (WAF) is a security tool created to guard online applications against many forms of assault, such as cross-site scripting, SQL injection, and other flaws. Between the web server and the client, it functions as an intermediary, removing harmful traffic and enforcing security regulations to stop attacks. To protect the integrity and accessibility of online services, WAFs identify and thwart threats in real-time. This highlights the crucial function of WAFs in protecting web applications from attacks and their advantages for businesses. It works by examining online traffic and removing requests or answers that might be hazardous, preventing unauthorized access to web servers and sensitive data. A WAF can be built as hardware or software and deployed in a variety of places, including as part of a load balancer or in front of web servers, for example. A WAF is fundamental to the security of online applications and their defense against the continuously changing threat landscape. A Web Application Firewall (WAF) is a security solution designed to protect web applications and websites from various types of cyber threats, including attacks such as cross-site scripting, SQL injection, and other application-layer attacks. A WAF works by analyzing incoming web traffic to a web application and blocking any malicious traffic that matches a predefined set of rules. This can be done through various techniques such as signature-based detection, behavioral analysis, and anomaly detection. WAFs can be deployed on-premises or in the cloud and can be configured to provide different levels of protection depending on the needs of the organization. Some WAFs can be integrated with other security solutions, such as IDS and SIEM tools, for better threat detection and response. Overall, a WAF is an important component of a comprehensive web application security strategy, helping organizations protect their web assets from cyber threats and ensure the security and privacy of their users. A WAF is

Figure 8.1: **Plagiarism Report**

Chapter 9

SOURCE CODE & POSTER PRESENTATION

9.1 Source Code

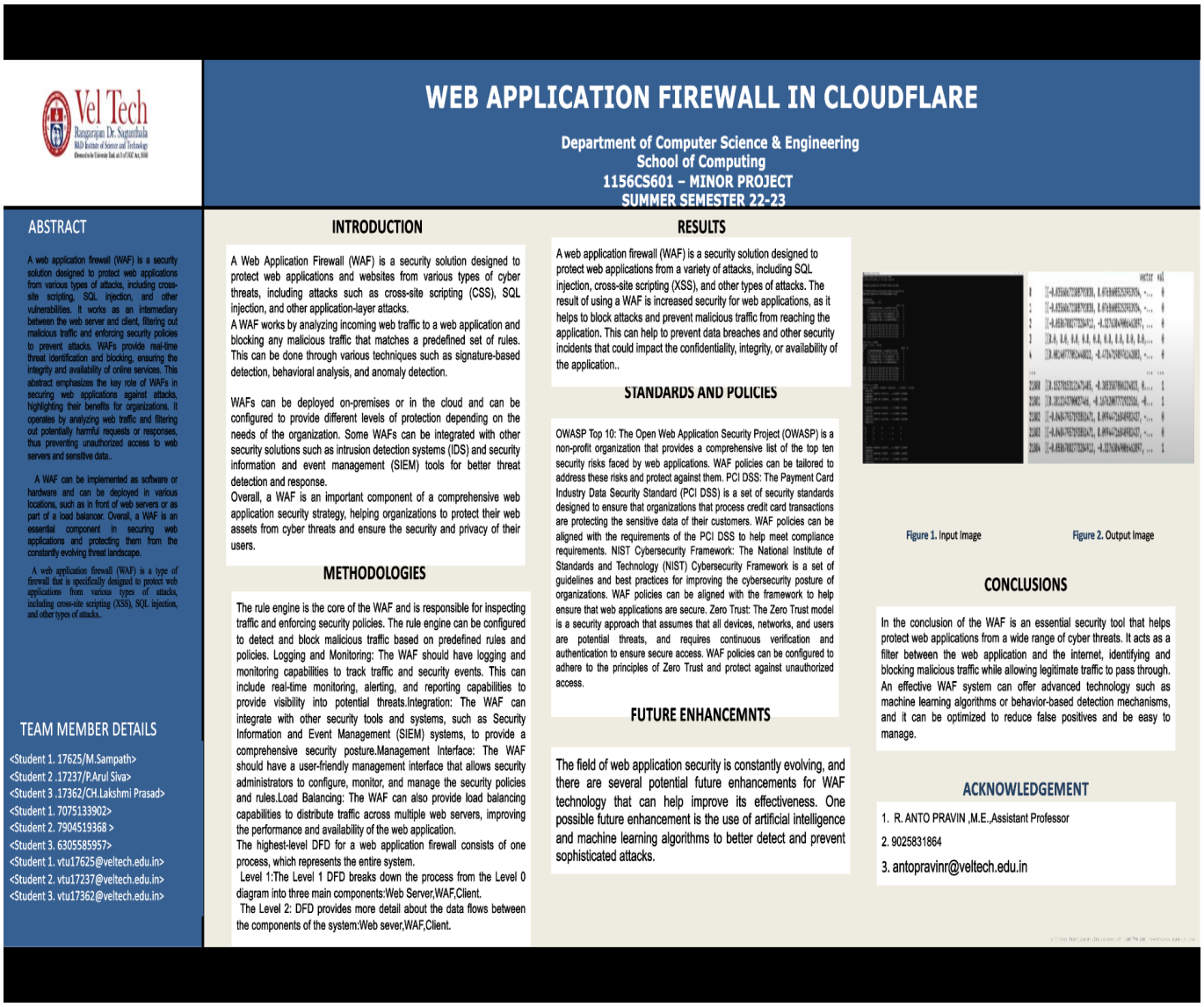
```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/contact', methods=['GET', 'POST'])
def contact():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        message = request.form['message']
        # Do something with the contact form data, e.g. send an email to the site owner
        return render_template('contact_success.html', name=name)
    else:
        return render_template('contact.html')

if __name__ == '__main__':
    app.run(debug=True)
\\
import re

def parse_request(request):
    # Parse the request to extract the HTTP method, path, and headers
    method, path, headers = re.search(r'^(.*?) (.*?) HTTP/1.1\r\n(.*?)\r\n\r\n', request, re.DOTALL)
        .groups()

    # Normalize the path by removing any query string parameters
    path = re.sub(r'\?.*$', '', path)
```

```
    # Normalize the headers by converting them to lowercase and removing any duplicate headers
    headers = dict((k.lower(), v) for k, v in [h.split(': ') for h in headers.split('\r\n') if ':' in h])

    return method, path, headers

import re

def detect_sql_injection(request):
    # Check the request path and query string parameters for common SQL injection patterns
    if re.search(r'\'|\b(select|insert|update|delete|union|drop|exec)\b', request['path'], re.IGNORECASE):
        return True
    for param in request['params']:
        if re.search(r'\'|\b(select|insert|update|delete|union|drop|exec)\b', param, re.IGNORECASE):
            return True
    return False

import re

def modify_response(response):
    # Add a Content-Security-Policy header to the response to prevent XSS attacks
    response['headers']['Content-Security-Policy'] = "default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline';"

    # Modify any links in the response to include a rel="noopener noreferrer" attribute to prevent tab hijacking
    response['body'] = re.sub(r'<a\s+(?:[^>]*?\s+)?href="([^"]*)"([^>]*)>', r'<a href="\1" rel="noopener noreferrer"\2>', response['body'])

    return response

 while (incoming_request)
2.      request = incoming_request()
3.      if (request_violates_rules(request))
4.          block_request()
5.      else
6.          forward_request()
7.      end if
8. end while

9. function request_violates_rules(request)
10.     for each rule in ruleset
11.         if (rule.matches(request))
12.             return true
13.         end if
14.     end for
15.     return false
16. end function
```

```
17. function block_request()
18.     send_response("Access denied", 403)
19.     end
20. end function

21. function forward_request()
22.     send_request(request)
23.     receive_response()
24.     send_response(response)
25.     end
26. end function

main code
// Import necessary libraries
import java.net.*;
import java.io.*;
import java.util.*;

// Define main class for the WAF
public class WebAppFirewall {

    // Define main method to intercept and analyze HTTP traffic
    public static void main(String[] args) throws IOException {

        // Create a socket to listen for incoming HTTP requests on port 80
        ServerSocket serverSocket = new ServerSocket(80);

        // Create an infinite loop to continuously process incoming requests
        while (true) {

            // Accept incoming HTTP request and establish a connection
            Socket clientSocket = serverSocket.accept();

            // Create input and output streams to read and write HTTP request and response data
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream
                ()));
            DataOutputStream out = new DataOutputStream(clientSocket.getOutputStream());

            // Read HTTP request data
            String requestLine = in.readLine();
            String headers = "";
            String line = in.readLine();
            while (line != null && line.length() > 0) {
                headers += line + "\n";
                line = in.readLine();
            }

            // Analyze HTTP request data to determine whether to allow or block traffic
            if (isMalicious(requestLine, headers)) {
```

```
                // Block traffic and return a 403 Forbidden response
                out.writeBytes("HTTP/1.1 403 Forbidden\r\n\r\n");
            } else {
                // Allow traffic and forward the HTTP request to the web server
                Socket webServerSocket = new Socket("localhost", 8080);
                DataOutputStream webOut = new DataOutputStream(webServerSocket.getOutputStream());
                BufferedReader webIn = new BufferedReader(new InputStreamReader(webServerSocket.
                    getInputStream()));
                webOut.writeBytes(requestLine + "\r\n" + headers + "\r\n");
                String response = "";
                line = webIn.readLine();
                while (line != null) {
                    response += line + "\n";
                    line = webIn.readLine();
                }
                out.writeBytes(response);
                webOut.close();
                webIn.close();
                webServerSocket.close();
            }

            // Log HTTP request and response data for future analysis and reporting
            log(requestLine, headers);

            // Close input and output streams and client socket connection
            in.close();
            out.close();
            clientSocket.close();
        }
    }

    // Define method to analyze HTTP request data and determine whether to allow or block traffic
    private static boolean isMalicious(String requestLine, String headers) {
        // Apply rules and policies to analyze HTTP request data and identify malicious traffic
        // Return true if traffic is malicious, false if it is benign
    }

    // Define method to log HTTP request and response data
    private static void log(String requestLine, String headers) {
        // Write log data to file or database for future analysis and reporting
        }
      }
     }
```

9.2 Poster Presentation



WEB APPLICATION FIREWALL IN CLOUDFLARE

Department of Computer Science & Engineering
School of Computing
1156CS601 – MINOR PROJECT
SUMMER SEMESTER 22-23

ABSTRACT

A web application firewall (WAF) is a security solution designed to protect web applications from various types of attacks, including cross-site scripting, SQL injection, and other vulnerabilities. It works as an intermediary between the web server and client, filtering out malicious traffic and enforcing security policies to prevent attacks. WAFs provide real-time threat identification and blocking, ensuring the integrity and availability of online services. This abstract emphasizes the key role of WAFs in securing web applications against attacks, highlighting their benefits for organizations. It operates by analyzing web traffic and filtering out potentially harmful requests or responses, thus preventing unauthorized access to web servers and sensitive data..

A WAF can be implemented as software or hardware and can be deployed in various locations, such as in front of web servers or as part of a load balancer. Overall, a WAF is an essential component in securing web applications and protecting them from the constantly evolving threat landscape.

A web application firewall (WAF) is a type of firewall that is specifically designed to protect web applications from various types of attacks, including cross-site scripting (XSS), SQL injection, and other types of attacks..

TEAM MEMBER DETAILS

<Student 1. 17625/M.Sampath>
<Student 2 .17237/P.Arul Siva>
<Student 3 .17362/CH.Lakshmi Prasad>
<Student 1. 7075133902>
<Student 2. 7904519368 >
<Student 3. 6305585957>
<Student 1. vtu17625@veltech.edu.in>
<Student 2. vtu17237@veltech.edu.in>
<Student 3. vtu17362@veltech.edu.in>

INTRODUCTION

A Web Application Firewall (WAF) is a security solution designed to protect web applications and websites from various types of cyber threats, including attacks such as cross-site scripting (CSS), SQL injection, and other application-layer attacks.
A WAF works by analyzing incoming web traffic to a web application and blocking any malicious traffic that matches a predefined set of rules. This can be done through various techniques such as signature-based detection, behavioral analysis, and anomaly detection.

WAFs can be deployed on-premises or in the cloud and can be configured to provide different levels of protection depending on the needs of the organization. Some WAFs can be integrated with other security solutions such as intrusion detection systems (IDS) and security information and event management (SIEM) tools for better threat detection and response.
Overall, a WAF is an important component of a comprehensive web application security strategy, helping organizations to protect their web assets from cyber threats and ensure the security and privacy of their users.

METHODOLOGIES

The rule engine is the core of the WAF and is responsible for inspecting traffic and enforcing security policies. The rule engine can be configured to detect and block malicious traffic based on predefined rules and policies. Logging and Monitoring: The WAF should have logging and monitoring capabilities to track traffic and security events. This can include real-time monitoring, alerting, and reporting capabilities to provide visibility into potential threats.Integration: The WAF can integrate with other security tools and systems, such as Security Information and Event Management (SIEM) systems, to provide a comprehensive security posture.Management Interface: The WAF should have a user-friendly management interface that allows security administrators to configure, monitor, and manage the security policies and rules.Load Balancing: The WAF can also provide load balancing capabilities to distribute traffic across multiple web servers, improving the performance and availability of the web application.
The highest-level DFD for a web application firewall consists of one process, which represents the entire system.
Level 1:The Level 1 DFD breaks down the process from the Level 0 diagram into three main components:Web Server,WAF,Client.
The Level 2: DFD provides more detail about the data flows between the components of the system:Web sever,WAF,Client.

RESULTS

A web application firewall (WAF) is a security solution designed to protect web applications from a variety of attacks, including SQL injection, cross-site scripting (XSS), and other types of attacks. The result of using a WAF is increased security for web applications, as it helps to block attacks and prevent malicious traffic from reaching the application. This can help to prevent data breaches and other security incidents that could impact the confidentiality, integrity, or availability of the application..

STANDARDS AND POLICIES

OWASP Top 10: The Open Web Application Security Project (OWASP) is a non-profit organization that provides a comprehensive list of the top ten security risks faced by web applications. WAF policies can be tailored to address these risks and protect against them. PCI DSS: The Payment Card Industry Data Security Standard (PCI DSS) is a set of security standards designed to ensure that organizations that process credit card transactions are protecting the sensitive data of their customers. WAF policies can be aligned with the requirements of the PCI DSS to help meet compliance requirements. NIST Cybersecurity Framework: The National Institute of Standards and Technology (NIST) Cybersecurity Framework is a set of guidelines and best practices for improving the cybersecurity posture of organizations. WAF policies can be aligned with the framework to help ensure that web applications are secure. Zero Trust: The Zero Trust model is a security approach that assumes that all devices, networks, and users are potential threats, and requires continuous verification and authentication to ensure secure access. WAF policies can be configured to adhere to the principles of Zero Trust and protect against unauthorized access.

FUTURE ENHANCEMNTS

The field of web application security is constantly evolving, and there are several potential future enhancements for WAF technology that can help improve its effectiveness. One possible future enhancement is the use of artificial intelligence and machine learning algorithms to better detect and prevent sophisticated attacks.

Figure 1. Input Image Figure 2. Output Image

CONCLUSIONS

In the conclusion of the WAF is an essential security tool that helps protect web applications from a wide range of cyber threats. It acts as a filter between the web application and the internet, identifying and blocking malicious traffic while allowing legitimate traffic to pass through. An effective WAF system can offer advanced technology such as machine learning algorithms or behavior-based detection mechanisms, and it can be optimized to reduce false positives and be easy to manage.

ACKNOWLEDGEMENT

1. R. ANTO PRAVIN ,M.E.,Assistant Professor
2. 9025831864
3. antopravinr@veltech.edu.in

Web application firewalls (WAFs) are security solutions that protect web applications from attacks and vulnerabilities. A WAF monitors and filters incoming and outgoing traffic to and from a web application, blocking any malicious traffic and preventing attacks such as SQL injection, cross-site scripting, and other common web application attacks.

References

[1] A. Sohail, M. Ghafoor, A. Farooq, F. Ullah. (2020). Effectiveness of web application firewalls in securing web applications: A systematic review. Computers Security, 94, 101875. https://doi.org/10.1016/j.cose.2020.101875

[2] Agarwal, M., Paliwal, R., Agarwal, S. (2021). Machine learning-based web application firewall for real-time threat detection. Journal of Network and Computer Applications, 175, 102910.

[3] Alghamdi, A. Abusukhon, A. Alhowaish. (2019). Evaluating the performance of web application firewalls in detecting and blocking attacks. Journal of Information Security and Applications, 48, 102398. https://doi.org/10.1016/j.jisa.2019.102398.

[4] R. S. Alhakami, A. F. Sheta, and H. S. Al-Kharusi, ”Web application firewall using artificial immune system,” 2016 IEEE/ACS 13th International Conference of Computer Systems and Applications (AICCSA), Agadir, Morocco, 2016, pp. 1-6.

[5] S. S. Gujrathi and S. V. Bonde, ”Evaluation of web application firewall performance and its comparison,” 2015 International Conference on Pervasive Computing (ICPC), Pune, India, 2015, pp. 1-6.

[6] N. M. Alharthi, T. B. Omer, and F. M. Alqahtani, ”Web application firewall based on machine learning algorithm,” 2017 13th International Conference on Innovations in Information Technology (IIT), Al Ain, United Arab Emirates, 2017, pp. 1-6.

[7] Y. A. Al-Dujaili, A. A. Abdul-Sattar, and M. A. Al-Sadoon, ”Web application firewall: A survey,” 2015 International Conference on Computing, Communica-

tion and Security (ICCCS), Ponnampet, India, 2015, pp. 1-6.

[8] S. S. Suhail and S. Venugopal, ”A review of web application firewalls,” 2016 International Conference on Electrical, Electronics, Communication, Computer, and Optimization Techniques (ICEECCOT), Mysore, India, 2016, pp. 347-351.

[9] M. A. B. A. Rahman and S. S. Saha, ”A hybrid approach of machine learning and rule-based web application firewall,” 2017 International Conference on Electrical, Computer and Communication Engineering (ECCE), Cox’s Bazar, Bangladesh, 2017, pp. 58-61.

[10] R. Gaurav and P. Khanna, ”Web application firewall: A comprehensive review,” 2017 International Conference on Computing, Communication and Automation (ICCCA), Greater Noida, India, 2017, pp. 1022-1026.